

1. The Mathematical Foundation

The core idea is to map a continuous range of values to a discrete set.

Linear Quantization (Affine Quantization)

The most common technique is mapping a floating-point value r to a quantized integer q using a **Scale (S)** and **Zero-point (Z)**.

The quantization formula is:

$$Q = \text{round} \left(\frac{r}{S} + Z \right)$$

To retrieve the approximate original value (Dequantization):

$$\tilde{r} = S(Q - Z)$$

- **Scale (S):** A positive float representing the step size of the quantization.
- **Zero-point (Z):** The integer value that represents the real value 0.0.

Symmetric vs. Asymmetric Quantization

- **Asymmetric:** Uses a non-zero Z to map the range $[\min, \max]$ to $[0, 255]$ (for INT8).
- **Symmetric:** Maps the range $[-\max|r|, \max|r|]$ to $[-127, 127]$. Here, Z is always 0, which simplifies the math and speeds up computation:

2. Core Quantization Strategies

Based on the provided research, we can categorize techniques into three main workflows:

1. **Post-Training Dynamic Quantization:** The scale and zero-point for activations are calculated on-the-fly during inference. It is easy to implement but adds slight latency.

2. **Post-Training Static Quantization:** Uses a "calibration dataset" to pre-calculate the distribution of activations. This is faster during inference than dynamic quantization.
3. **Quantization-Aware Training (QAT):** Models are trained with "fake quantization" layers. The model learns to compensate for the rounding errors, resulting in the highest accuracy at low bit-widths (e.g., 2-bit or 4-bit).

3. Implementation Example: 4-bit Quantization

Using the `bitsandbytes` library (as referenced in modern LLM practitioners' guides), we can load a massive model like LLaMA on a single consumer GPU by compressing it into **4-bit NormalFloat (NF4)**.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

model_id = "meta-llama/Llama-2-7b-hf"

# 1. Configure the 4-bit quantization (NF4)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)

# 2. Load the model with quantization
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map="auto"
)

# 3. Check memory savings
print(f"Model memory footprint: {model.get_memory_footprint() / 1e9:.2f} GB")
```

4. Advanced Techniques (Non-Uniform Quantization)

Yacine Bouaouni's guide also highlights that while linear methods are popular, **Non-Uniform Quantization** can preserve more information:

Here is a **clean, academic version suitable for a report**:

Quantization Methods

Quantization methods are used to reduce the numerical precision of model parameters while maintaining acceptable accuracy. The choice of method depends on the data distribution and the requirements of the target application. This section describes three major quantization approaches: Linear Quantization, Logarithmic Quantization, and Non-Uniform Quantization

1. Linear Quantization

Linear quantization is the most widely used and computationally efficient technique. It maps floating-point values from a continuous range into a smaller, fixed integer range using a linear scaling factor.

For example, if weights lie in the range:

$[-r, r]$

they can be scaled into:

$[-128, 127]$

for 8-bit signed integer representation.

This method applies a uniform step size across the entire value range. It is simple to implement and highly efficient for hardware acceleration. However, it may not preserve accuracy well if the data distribution is highly concentrated in specific regions (e.g., near zero).

2. Logarithmic Quantization

Logarithmic quantization uses a logarithmic scale rather than a uniform linear scale. In this approach, smaller values are represented with finer precision, while larger values are represented with coarser precision.

This method is particularly effective when the data has a wide dynamic range or follows an exponential or bell-shaped distribution. Since neural network weights are often concentrated around zero, logarithmic scaling can preserve important small-magnitude values more effectively than linear quantization.

3. Non-Uniform (Non-Linear) Quantization

Non-uniform quantization allocates quantization levels unevenly across the value range. Instead of using equal intervals, it adapts the quantization boundaries according to the data distribution. This approach aims to minimize quantization error by assigning more precision to regions where data is dense.

Two common implementations include:

3.1 Lloyd–Max Quantization

Lloyd–Max quantization is an iterative optimization algorithm that minimizes the mean squared quantization error (MSE). It adjusts quantization levels and decision boundaries repeatedly until convergence. This method is commonly used in scalar signal quantization.

3.2 K-Means Quantization

K-means quantization uses clustering to determine optimal quantization levels. The algorithm groups weight values into K clusters and replaces each value with the index of its nearest cluster centroid. This approach reduces representation error and is particularly effective for low-bit quantization (e.g., 2-bit or 4-bit systems).

Summary Table: Impact of Quantization

Technique	Bit Depth	Memory Reduction	Accuracy Trade-off
FP32	32-bit	1x (Baseline)	None
FP16 / BF16	16-bit	2x	Negligible
INT8	8-bit	4x	Very Low
NF4 / INT4	4-bit	8x	Low to Moderate

GPTQ / AWQ	2/3/4-bit	Up to 12x	Varies (SOTA for LLMs)
-------------------	-----------	-----------	------------------------

By implementing these techniques, we can "shrink" models like LLaMA to fit on mobile devices or standard laptops without losing the core reasoning capabilities that make them useful.